

SIMATS ENGINEERING

ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks: 100 Duration: 1 Hour

Industry Problem

Code of Conduct:

I J.Goutham Kumar Reddy (192311140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J.Goutham Kumar Reddy

ASSESSMENT TOOLS

1. Design and implement a C-based validation module that simulates a Deterministic Finite Automaton (DFA) to verify input strings in a real-time system. The module should accept only those strings that begin with the character 'a' and end with the character 'a', ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is Accepted or Rejected

2. Design a Push Down Automata that accepts the language $L = \{ w \mid w \text{ contains } (0+1)^* \text{ and } n_0(w) = n_1(w) \}$
 $n_0(w)$ is the number of 0's in w $n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice that of a's.

$$L = \{ a^n b^{2n} \mid n \geq 1 \}$$

4. Develop a C-based computational module to determine the ϵ -closure of all states in a Non Deterministic Finite Automaton (NFA) with ϵ -transitions, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a C-based input validation module that determines whether a given binary string conforms to a specified Context-Free Grammar (CFG) used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

ASSESSMENT TOOLS — RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5)	Level 3 – Proficient (4)	Level 2 – Developing (2–3)	Level 1 – Beginning (0–1)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

Question 1 — DFA for strings that begin with 'a' and end with 'a'

Design

The DFA accepts strings over {a, b} that begin with 'a' and end with 'a'.

States: S0 (start), S1 (first char was 'a', last char read is 'a' — accepting), S2 (first char was 'a', last char read is 'b'), S3 (trap/dead state, used once the first character is 'b').

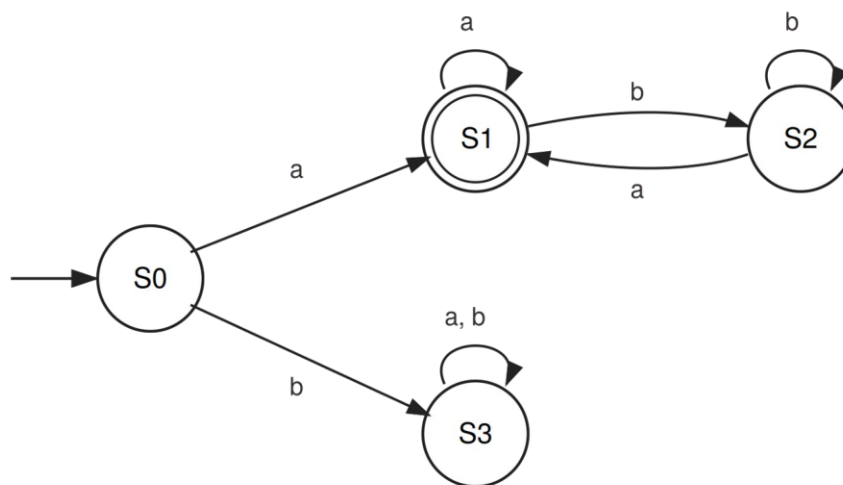
Transition table:

- S0 --a--> S1 S0 --b--> S3
- S1 --a--> S1 S1 --b--> S2
- S2 --a--> S1 S2 --b--> S2
- S3 --a--> S3 S3 --b--> S3

Start state: S0. Accepting state: S1 only.

The empty string is rejected because S0 is not an accepting state.

State Transition Diagram



Start state: S0 | Accepting state: S1 (double circle)

C Program

```
#include <stdio.h>
#include <string.h>

typedef enum { S0, S1, S2, S3 } State;

int main() {
    char
    input[1000];
    printf("Enter input string (a/b only): ");
    scanf("%999s", input);
    int len = strlen(input);
    State state = S0;
    for (int i = 0; i < len; i++) {
        char ch = input[i];
```

```

        switch (state) {
            case S0:
                state = (ch == 'a') ? S1 : S3;
                break;
            case S1:
                state = (ch == 'a') ? S1 : S2;
                break;
            case S2:
                state = (ch == 'a') ? S1 : S2;
                break;
            case S3:
                state = S3; /* dead state: stays trapped */
                break;
        }
    }
    if (state == S1)
        printf("Accepted\n");
    else
        printf("Rejected\n");
    return 0;
}

```

Question 2 — PDA for $L = \{ w \in (0+1)^* \mid n_0(w) = n_1(w) \}$

Design

Language: $L = \{ w \in (0+1)^* \mid n_0(w) = n_1(w) \}$, i.e. strings with an equal number of 0's and 1's.

A non-deterministic PDA that accepts by empty stack is used. Stack alphabet: $\{0, 1, Z_0\}$, Z_0 is the bottom-of-stack marker.

Single state q_0 is sufficient (all transitions loop on q_0), plus a final state q_f reached when the stack empties.

Idea: every symbol read either matches (cancels) the symbol on top of the stack, or is pushed if it does not match / stack shows only Z_0 .

Transitions ($\delta(\text{state}, \text{input}, \text{stack-top}) = (\text{state}, \text{push-string})$):

- $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$ — push 0 when stack is empty of counters
- $\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$ — push 1 when stack is empty of counters
- $\delta(q_0, 0, 0) = (q_0, 00)$ — another 0 while 0's are on top: push 0
- $\delta(q_0, 1, 1) = (q_0, 11)$ — another 1 while 1's are on top: push 1
- $\delta(q_0, 0, 1) = (q_0, \epsilon)$ — 0 cancels a 1 on the stack: pop
- $\delta(q_0, 1, 0) = (q_0, \epsilon)$ — 1 cancels a 0 on the stack: pop
- $\delta(q_0, \epsilon, Z_0) = (q_f, \epsilon)$ — accept by empty stack when only Z_0 is left

At the end of the input, if the stack contains only Z_0 , the counts of 0's and 1's were equal at every point they cancelled, so $n_0(w) = n_1(w)$ and the PDA accepts.

PDA Diagram



Accept by empty stack: pops Z0 and moves to q_f only when the stack held nothing but Z0

Question 3 — PDA for $L = \{ a^n b^{2n} \mid n \geq 1 \}$

Design

Language: $L = \{ a^n b^{2n} \mid n \geq 1 \}$, i.e. n a's followed by exactly $2n$ b's.

PDA (accept by empty stack / final state), stack alphabet $\{X, Z_0\}$.

States: q_0 (reading a's, push phase), q_1 (reading b's, pop phase), q_f (accept state).

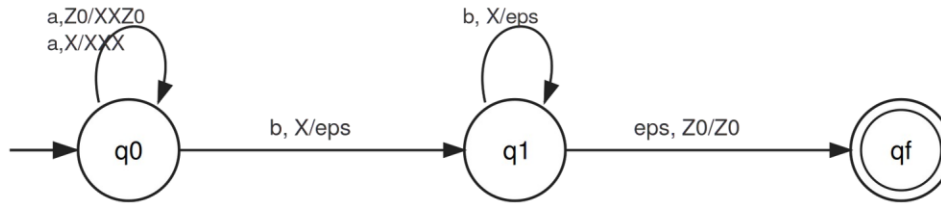
Idea: for every 'a' read, push two X's onto the stack (so the stack height records $2 \times$ number of a's). For every 'b' read, pop exactly one X. If the input is well formed, the stack becomes empty (down to Z_0) exactly when the last b is read.

Transitions:

- $\delta(q_0, a, Z_0) = (q_0, XXZ_0)$ — first a pushes two X's on top of Z_0
- $\delta(q_0, a, X) = (q_0, XXX)$ — every subsequent a pushes two more X's
- $\delta(q_0, b, X) = (q_1, \epsilon)$ — first b switches to pop phase and pops one X
- $\delta(q_1, b, X) = (q_1, \epsilon)$ — every subsequent b pops one X
- $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$ — accept when only the bottom marker remains

Since $n \geq 1$, at least one 'a' must be read before the machine can move to q_1 , guaranteeing at least 2 b's are required, matching $a^n b^{2n}$ for $n \geq 1$.

PDA Diagram



q0: push 2 X's per 'a'. q1: pop 1 X per 'b'. qf: accept when only Z0 remains.

Question 4 — ϵ -closure of all states in an NFA with ϵ -transitions

Design

ϵ -closure(state) is the set of all states reachable from "state" using zero or more ϵ -transitions. It is computed for every state of the NFA using a graph traversal (DFS/BFS) over the ϵ -transition edges only.

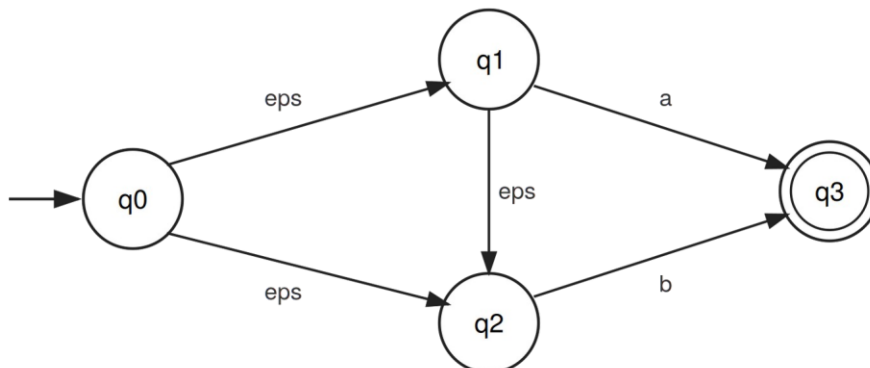
Algorithm:

- 1. Represent the NFA as an adjacency structure: $\text{eps}[i]$ = list of states reachable from state i on a single ϵ move.
- 2. For each state i , initialise $\text{closure} = \{i\}$.
- 3. Use a stack/queue; push i . While the stack is not empty, pop a state s , and for every state t in $\text{eps}[s]$ not already in closure , add t to closure and push t .
- 4. Repeat for all states and print the resulting closure sets.

In the example NFA diagram: $\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$, $\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$, $\epsilon\text{-closure}(q_2) = \{q_2\}$, $\epsilon\text{-closure}(q_3) = \{q_3\}$.

Example NFA with ϵ -transitions

$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$ $\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$ $\epsilon\text{-closure}(q_2) = \{q_2\}$ $\epsilon\text{-closure}(q_3) = \{q_3\}$



C Program

```
#include <stdio.h>

#define MAX 20

int n; /* number of states */
int epsAdj[MAX][MAX]; /* epsAdj[i][j] = 1 if i --eps--> j */
int visited[MAX];

void dfsClosure(int state, int closure[]) {
    if (visited[state]) return;
    visited[state] = 1; closure[state] = 1;
    for (int j = 0; j < n; j++)
        if (epsAdj[state][j] && !visited[j])
            dfsClosure(j, closure);
}

int main() {
    int edges;

    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter number of epsilon-transitions: ");
    scanf("%d", &edges);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            epsAdj[i][j] = 0;
    printf("Enter each epsilon-transition as: from to\n");
    for (int e = 0; e < edges; e++) {
        int a, b;
        scanf("%d %d", &a, &b);
        epsAdj[a][b] = 1;
    }
    for (int s = 0; s < n; s++) {
        int closure[MAX] = {0};
        for (int k = 0; k < n; k++) visited[k] = 0;
        dfsClosure(s, closure);
        printf("epsilon-closure(q%d) = { ", s);
        for (int k = 0; k < n; k++)
            if (closure[k]) printf("q%d ", k);
        printf("}\n");
    }
    return 0;
}
```

Question 5 — CFG-based binary string validation ($S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \epsilon$)

Design

Grammar: $S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \text{epsilon}$

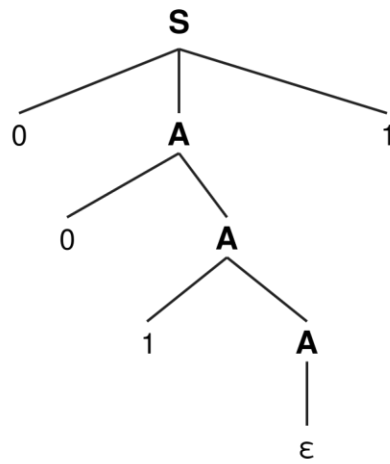
S always produces a leading 0, then A generates zero or more symbols from $\{0,1\}$ (since A can expand to $0A$, $1A$ or epsilon), and finally the fixed trailing 1 from the S -rule closes the string.

So the language generated is: $L(G) = \{ 0x1 \mid x \text{ is any string over } \{0,1\}, \text{ including the empty string } \}$, i.e. every string that starts with 0, ends with 1, and has length ≥ 2 .

The C module below performs a direct top-down (recursive-descent) simulation of the grammar's derivation to validate a given binary string against $S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \text{epsilon}$: it consumes the fixed

leading 0 and trailing 1 required by S, and lets A consume everything in between, matching the grammar rule by rule.

Example Parse Tree (for input "0011")



Derivation for input "0011": $S \rightarrow 0A1 \rightarrow 00A1 \rightarrow 001A1 \rightarrow 0011$

C Program

```
#include <stdio.h>
#include <string.h>

char input[1000];
int pos;
int len;

/* A -> 0 A | 1 A | epsilon */
/* Consumes symbols greedily until only the final '1' required */
/* by S is left, then lets S consume that closing '1'. */
void A() {
    if (pos < len - 1 && (input[pos] == '0' || input[pos] == '1')) {
        pos++;      /* matched 0A or 1A */
        A();        /* recurse for the rest of A */
    }
    /* else: A -> epsilon, nothing more to consume */
}

/* S -> 0 A 1 */
int S() {
    if (pos < len && input[pos] == '0') {
        pos++;      /* matched leading 0 */
        A();        /* matched A */
    }
    if (pos < len && input[pos] == '1') {
        pos++;      /* matched trailing 1 */
        return (pos == len); /* valid only if whole string consumed */
    }
    return 0;
}

int main() {
    printf("Enter a binary string: ");
```

```
    scanf("%999s", input);
    len = strlen(input);
    pos = 0;
    /* reject anything that is not purely 0's and 1's */
    for (int i = 0; i < len; i++) {
        if (input[i] != '0' && input[i] != '1') {
            printf("Rejected (invalid symbol)\n");
            return 0;
        }
    }
    if (len >= 2 && S())
        printf("Accepted\n");
    else
        printf("Rejected\n");
    return 0;
}
```